

# Cómo decide SmartCourier AI

Funcionamiento implementado: aceptación, espera, rechazo y agrupación. Documento basado en los archivos actuales del proyecto, incluidos los cambios locales sin commit; referencia Git: 2a28a5b.

**La regla principal:** si está libre, SmartCourier estima la ganancia neta de recoger y entregar, verifica restricciones y combina seis factores en un puntaje de 0 a 100. Si ya está entregando, evalúa si puede agregar el pedido a su ruta. [S1-S4]

## 1. Primero importa si el repartidor está ocupado

| Situación                | Camino que ejecuta el motor                                                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Libre o reposicionándose | Construye la ruta posición actual → recolección → entrega. Calcula economía, restricciones y puntaje. Un pedido aceptado reemplaza el plan de reposicionamiento. |
| Con una entrega activa   | Entra en tryBatch(): compara el recorrido pendiente con una ruta que incluye el nuevo pedido. No ejecuta el puntaje de aceptación individual.                    |

## 2. Resultado cuando está libre

| Condición, en este orden              | Decisión | Qué ocurre                               |
|---------------------------------------|----------|------------------------------------------|
| Una o más restricciones incumplidas   | REJECT   | Declina, aunque el puntaje sea alto.     |
| Sin restricciones y puntaje ≥ 40      | ACCEPT   | Asigna el pedido y comienza la ruta.     |
| Sin restricciones y 25 ≤ puntaje < 40 | WAIT     | Declina esta oferta y espera otra mejor. |
| Sin restricciones y puntaje < 25      | REJECT   | Declina por baja conveniencia.           |

**WAIT no reserva ni reevalúa el pedido.** En el motor actual incrementa el contador de rechazados, igual que REJECT. La distinción expresa el motivo de la decisión. No calcula la probabilidad real de que llegue una oferta mejor. [S1, S2]

## 3. Qué significa “conviene”

Es una clasificación heurística: las reglas consideran atractivo el pedido bajo sus estimaciones y límites. Puede rechazar un pedido con ganancia positiva por consumir demasiado tiempo o recorrido. El puntaje no es un porcentaje de éxito ni una garantía de maximizar el ingreso del turno.

**Otros resultados:** BATCH acepta un pedido adicional compatible; REPOSITION mueve al agente hacia una zona con mayor demanda si se habilita esa función. El reposicionamiento está desactivado por defecto. [S2, S4, S5]

# Qué información usa

Las variables guardadas o mostradas en pantalla no necesariamente intervienen en la decisión. Esta página distingue su uso real.

| Entrada o estado                                                     | Uso exacto                                                                                                                                                        |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>final_payment</b> · MXN                                           | Ingreso bruto ofrecido; entra en ganancia neta y rentabilidad marginal.                                                                                           |
| <b>Posición del agente</b><br>pickup_lat/lng, dropoff_lat/lng        | Definen recorridos, duración y distancia al negocio. Las coordenadas de recolección también se comparan con la zona de trabajo.                                   |
| <b>distance_km</b><br>estimated_time_minutes                         | Distancia y duración de la entrega ya ajustadas al publicar el pedido. Se reutilizan en el tramo directo recolección → entrega; también son entradas de Baseline. |
| <b>estimated_preparation_minutes</b><br>created_at_simulation_second | Determinan cuándo está listo el pedido y cuánto hay que esperar. La hora de creación define el filtro nocturno.                                                   |
| <b>traffic_level</b><br>cierres viales activos                       | Modifican tiempos y, para cierres, distancias de los tramos calculados. No suman puntos directamente.                                                             |
| <b>destination_demand</b>                                            | LOW, MEDIUM, HIGH o VERY_HIGH. Aporta hasta 15 puntos; proviene de reglas de zona/hora o de un evento.                                                            |
| <b>vehicle_type</b>                                                  | Selecciona costo por km: bike, motorcycle o car. El costo predeterminado corresponde a motocicleta.                                                               |
| <b>package_weight / bag_max_weight</b>                               | Veto por peso individual al estar libre; peso combinado al agrupar, si existe límite configurado.                                                                 |
| <b>Zona y límite nocturno</b>                                        | work_zone_center_lat/lng + work_zone_radius_km;<br>night_distance_limit_km. Son filtros de aceptación individual.                                                 |
| <b>Plan y pedidos activos</b>                                        | Habilitan agrupación y permiten medir km extra, minutos extra y retrasos de entregas comprometidas.                                                               |

## Campos que no son criterios activos de SmartCourier

Los mínimos de pago, pago/minuto, pago/km y distancia máxima configurados se aplican a **Baseline**, no al puntaje SmartCourier. No se usan como filtros activos las dimensiones/volumen de mochila, zonas preferidas o evitadas, avoid\_configured\_zones ni los horarios de turno guardados. [S1, S2, S6, S10]

package\_size, priority y expiration\_seconds no se consultan para aceptar, esperar o agrupar. Surge y el impulso de urgencia sí pueden cambiar final\_payment antes de evaluar; no hay un peso adicional por urgencia. Las preferencias se cargan al construir el motor. [S2, S7, S10]

# Cómo calcula lo que deja un pedido

La estimación incluye el viaje vacío al negocio, la espera restante y la entrega. “Neto” aquí significa pago menos costo operativo por km; no descuenta impuestos ni costos fijos.

## 1. Construye la línea de tiempo [S3, S7]

listo = segundo de creación + preparación × 60  
llegada = inicio de la ruta + segundos del trayecto al negocio  
espera = max(0, listo - llegada) / 60  
**T = max((fin de entrega - inicio de ruta) / 60, 1/60)**

Cada tramo usa max(0, Math.round(duración\_en\_minutos × 60)) segundos. La preparación transcurre mientras el repartidor llega: no se suma completa si parte de ella ya pasó. No hay un tiempo adicional de servicio al entregar.

## 2. Calcula distancia, costo y rentabilidad [S3]

**K** = km al negocio + km de entrega  
**C** = K × costo por km del vehículo  
**N** = pago final - C  
**ganancia/minuto** = N / T  
**ganancia/km** = N / K; si K = 0, el código utiliza N

| Vehículo                 | Costo operativo codificado                 |
|--------------------------|--------------------------------------------|
| Bicicleta · bike         | \$0.40 / km                                |
| Motocicleta · motorcycle | \$2.00 / km · también el valor de respaldo |
| Automóvil · car          | \$4.50 / km                                |

Los costos son constantes del simulador, no precios de mercado verificados. offerEconomics calcula con valores internos y devuelve cada métrica con dos decimales; el puntaje y el veto por ganancia usan esas métricas redondeadas. No deben reconstruirse los cocientes a partir de otros valores ya redondeados si se busca reproducir todos los casos frontera.

## 3. Cómo obtiene rutas y pago [S7, S8]

**Rutas:** usa OSRM con perfil driving cuando está configurado. Si falta o falla tras un timeout de 5 s, estima km = Haversine × 1.38 y minutos = km / 45 × 60. El tipo de vehículo cambia el costo, pero actualmente no cambia el perfil ni la velocidad del estimador.

**Tráfico:** LOW ×1.00, MEDIUM ×1.15, HIGH ×1.35, SEVERE ×1.60 sobre duración. Un cierre aplica ×1.60 a km y duración base; no calcula un desvío por una calle específica.

**Pago simulado:** base = redondear2(km de ruta base × tarifa/km + cuota fija); final = redondear2(base × surge × impulso urgente). El generador usa tarifa de 9 a 14 MXN/km, cuota de 15 a 25 MXN e impulso 1.30 solo para pedidos forzados urgentes. El cierre no aumenta la base de pago.

# La fórmula de aceptación

Los pesos suman 100 puntos. Primero se calculan restricciones y factores; al resolver la decisión, cualquier restricción tiene prioridad sobre el puntaje. [S1]

**clip(x)** = min(1, max(0, x))  
**S** = Math.round(suma de los seis aportes)  
**N** = ganancia neta; **T** = minutos totales; **Dp** = km al negocio;  
**r** = ganancia por minuto; **q** = ganancia por km.

| Factor              | Aporte exacto                  | Máximo |
|---------------------|--------------------------------|--------|
| Ganancia por minuto | 35 × clip((r - 0.50) / 3.50)   | 35     |
| Ganancia neta       | 15 × clip(N / 120)             | 15     |
| Ganancia por km     | 15 × clip((q - 2) / 10)        | 15     |
| Cercanía al negocio | 10 × (1 - clip(Dp / 8))        | 10     |
| Tiempo total        | 10 × (1 - clip((T - 15) / 45)) | 10     |
| Demanda del destino | 15 × f; f = 0 / 0.4 / 0.7 / 1  | 15     |

Demanda: LOW = 0 puntos; MEDIUM = 6; HIGH = 10.5; VERY\_HIGH = 15. Un valor desconocido usa f = 0.4. Los rangos se saturan: por ejemplo, superar \$4/min no aporta más de 35 puntos. Un factor se muestra como positivo si alcanza al menos 40% de su peso; si queda por debajo se etiqueta negativo, pero no resta puntos.

## Los cuatro vetos al evaluar un pedido individual

| Código                        | Condición exacta que provoca REJECT                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------|
| BAG_CAPACITY_EXCEEDED         | Peso del pedido > bag_max_weight, si se configuró.                                                                 |
| NIGHT_DISTANCE_LIMIT_EXCEEDED | Hora ≥ 22 o < 6 y distance_km > night_distance_limit_km, si se configuró. Solo distancia de entrega.               |
| OUTSIDE_WORK_ZONE             | Haversine(centro, recolección) > radio, si están configurados centro y radio. No verifica destino ni toda la ruta. |
| NEGATIVE_PROFIT               | netProfit ≤ 0, incluyendo exactamente cero.                                                                        |

Hora simulada = floor(12 + segundo\_de\_creación / 3600) % 24. Igualar el peso, radio o distancia nocturna máxima no activa el veto: la comparación es estrictamente >. Sin configuración válida de una restricción opcional, esa comprobación no se aplica. [S1, S9]

# Cuándo conviene y cuándo se declina

Casos didácticos ejecutados con buildTimeline, offerEconomics y evaluateSmartCourier del proyecto. Todos usan motocicleta a \$2/km; no son ofertas reales.

| Dato                       | A · atractivo | B · intermedio | C · débil       |
|----------------------------|---------------|----------------|-----------------|
| Pago final                 | \$100         | \$70           | \$45            |
| Km al negocio / entrega    | 1 / 4         | 5 / 5          | 8 / 6           |
| Min al negocio / entrega   | 4 / 16        | 20 / 20        | 25 / 25         |
| Preparación desde creación | 12 min        | 10 min         | 10 min          |
| Espera restante            | 8 min         | 0 min          | 0 min           |
| Tiempo / km totales        | 28 min / 5 km | 40 min / 10 km | 50 min / 14 km  |
| Costo / ganancia neta      | \$10 / \$90   | \$20 / \$50    | \$28 / \$17     |
| Ganancia/min · ganancia/km | \$3.21 · \$18 | \$1.25 · \$5   | \$0.34 · \$1.21 |
| Demanda                    | HIGH          | MEDIUM         | LOW             |
| Puntaje / decisión         | 80 · ACCEPT   | 32 · WAIT      | 4 · REJECT      |

## Desglose del caso A

Ganancia/min: 27.10; ganancia neta: 11.25; ganancia/km: 15; cercanía: 8.75; tiempo total: 7.11; demanda: 10.50. La suma interna es aproximadamente 79.71 y Math.round produce **80**. Sin restricciones, acepta.

## La utilidad positiva no basta

El caso B dejaría \$50 netos, pero solo \$1.25/min: sus 32 puntos llevan a WAIT. El caso C también deja dinero (\$17), pero sus 4 puntos llevan a REJECT. En ambos casos el pedido se declina y aumenta el contador de rechazados.

## Un veto gana aunque el pedido sea atractivo

**Caso D:** repetir A con paquete de 6 kg y capacidad configurada de 5 kg conserva el puntaje 80, pero termina en REJECT por BAG\_CAPACITY\_EXCEEDED. **Caso E:** recorrer 10 km en moto para cobrar \$20 deja \$0 netos; termina en REJECT por NEGATIVE\_PROFIT.

**Fronteras del puntaje:** S = 40 ya acepta; S = 25 ya es WAIT; S = 24 rechaza. La comparación utiliza el entero redondeado: una suma interna de 39.5 produce 40 y una de 24.5 produce 25, si no hay restricciones.

# Cómo decide agrupar pedidos

BATCH compara incrementos respecto de lo que falta de la ruta actual. No exige 40 puntos ni utiliza la demanda del destino como factor. [S2, S4]

## 1. Construye y compara recorridos

Toma las paradas pendientes y añade recolección y entrega del nuevo pedido. bestSequence enumera secuencias válidas que recogen cada pedido antes de entregarlo; elige la de menor hora estimada de fin y, si empatan, la de menos km. La selección usa rutas aproximadas. Luego calcula la ruta candidata y recalcula la actual desde la misma posición con el servicio de rutas. [S2, S7]

$$\Delta K = \text{km restantes candidatos} - \text{km restantes actuales}$$
$$\Delta T = (\text{fin candidato} - \text{fin actual}) / 60$$
$$\Delta C = \Delta K \times \text{costo por km}$$
$$\Delta N = \text{pago del nuevo pedido} - \Delta C$$
$$\text{rentabilidad marginal} = \Delta N / \max(\Delta T, 1)$$
$$\text{retraso máximo} = \max(0, \text{retraso de cada entrega ya comprometida})$$

| Debe cumplir todas                           | Límite predeterminado                              |
|----------------------------------------------|----------------------------------------------------|
| Cantidad de pedidos activos antes de agregar | Menos de 2; quedan como máximo 2 en ruta.          |
| Suma de pesos activos + peso nuevo           | $\leq$ capacidad en kg, si se configuró.           |
| Retraso máximo de entregas comprometidas     | $\leq$ 15 minutos.                                 |
| Tiempo y distancia adicionales               | $\Delta T \leq 20$ minutos y $\Delta K \leq 6$ km. |
| Ganancia neta por minuto adicional           | $\geq$ \$3.00 por minuto.                          |

Si pasa todos los filtros, compatible = true y el motor registra BATCH; aumenta aceptados y agrupados y reemplaza la ruta. Si falla, registra REJECT con AGENT\_BUSY y los motivos específicos. No existe WAIT para este camino. Los límites se comparan antes del redondeo de los datos que se muestran.

## Ejemplo de agrupación

Ruta actual: 6 km pendientes y fin en minuto 30. Candidata: 8 km y fin en minuto 40; retrasa la entrega previa 5 min. Nuevo pago: \$70.  $\Delta K = 2$ ,  $\Delta T = 10$ ,  $\Delta C = \$4$  y  $\Delta N = \$66$ ; deja **\$6.60/min extra**. Con capacidad suficiente y un solo pedido activo, produce BATCH.

## Diferencias que afectan la decisión

El evaluador de agrupación verifica peso combinado, pero no vuelve a ejecutar los filtros de zona de trabajo o distancia nocturna. El retraso se compara con la ruta actual recalculada, no con una hora de entrega prometida originalmente. Además, una vez seleccionada la secuencia aproximada, no explora otra si la evaluación final la rechaza. Son oportunidades de mejora, no reglas ya implementadas.

# Detalles técnicos importantes

El camino principal es Node.js: SimulationEngine → SimulationCore → evaluadores y routePlanner. Las decisiones centrales son reglas deterministas; no interviene un modelo de lenguaje ni un modelo entrenado.

## Baseline es la referencia simple [S6]

Si está ocupado, rechaza. Si está libre, acepta solo si se cumplen las cuatro reglas: pago  $\geq$  mínimo (respaldo \$0), pago/tiempo de entrega  $\geq$  \$3/min, pago/km de entrega  $\geq$  \$8/km y distancia de entrega  $\leq$  15 km. Las preferencias del usuario sustituyen estos respaldos. Decide con el pago bruto y la entrega; cuando acepta, el motor sí le cobra el recorrido al negocio, la espera y los mismos costos que a SmartCourier.

Existe también un controlador independiente `baseline.controller.js` que solo compara pago/tiempo  $\geq$  3. No es el evaluador de cuatro reglas que usa la simulación. El servicio Python con OR-Tools expone optimización por API, pero SimulationCore no lo llama: agrupa con `bestSequence` en JavaScript. [S2, S6, S13]

## Demanda y reposicionamiento [S5, S9]

Para la demanda geográfica se usa la zona más cercana; si el destino está a más de 4 km de su centro, su demanda es LOW. Dentro de ese rango: valor = clip(base horaria + atractivo de zona). Base: 0.8 de 18:00 a antes de 21:00, 0.6 de 12:00 a antes de 14:00 y 0.3 el resto. Atractivos: Centro +0.3, San Pedro +0.2, San Nicolás +0.1, Guadalupe 0, Cumbres -0.1 y Apodaca -0.2.

Conversión: valor  $\geq$  0.75 → VERY\_HIGH;  $\geq$  0.50 → HIGH;  $\geq$  0.25 → MEDIUM; menor → LOW. El pedido conserva la demanda calculada al crearse, salvo el modificador activo al publicarlo; no pronostica demanda a la hora estimada de llegada. Los eventos HIGH\_DEMAND y LOW\_DEMAND pueden sustituirla. [S7, S9]

Reposicionamiento: desactivado por defecto. Al habilitarlo se revisa cada 10 min simulados, solo sin plan. Si la zona actual ya es HIGH/VERY\_HIGH espera; de lo contrario busca una zona con mayor nivel a  $\leq$  10 km en línea recta y dentro de la zona de trabajo, si se configuró. Ordena por demanda y luego cercanía; no compara el beneficio esperado con su costo.

## Reloj, cobro y reproducibilidad [S2, S8, S10]

Ambos agentes reciben la misma secuencia y empiezan en la misma ubicación, pero mantienen posiciones y contadores separados. Los eventos se procesan en segundos simulados exactos. A 1x transcurren 60 segundos simulados por segundo real; 2x, 5x y 10x cambian la reproducción, no la regla de decisión. El motor serializa ticks, eventos y cierre para evitar modificaciones simultáneas.

Los costos se acumulan conforme se recorren km; el ingreso se cobra al entregar, no al aceptar. Al terminar el turno, completa las entregas activas aunque acaben después del horario. Las tasas por minuto de turno pueden incluir esos cobros con la duración nominal como denominador. La caché comparte hasta 2,000 rutas; una seed por sí sola no fija cambios externos de OSRM o del entorno.

## Límites observados [S2, S10, S12]

Restaurar tras reiniciar el proceso recupera contadores y posición, pero descarta rutas en curso. Tráfico y cierres no disparan un recálculo global inmediato de planes activos; el tramo directo de entrega reutiliza sus valores publicados. El “qué habría pasado si...” de la interfaz solo cambia el signo de la economía del pedido: no reproduce el resto del turno ni estima ofertas futuras.

# Evidencia y trazabilidad

Fecha de verificación: 12 de septiembre de 2026. Los resultados corresponden al árbol de trabajo local revisado, no únicamente al commit base ni a datos de operación real.

## Comprobaciones ejecutadas

**Pruebas:** la ejecución general pasó 45 de 46; la única prueba restante no pudo conectarse a PostgreSQL por el bloqueo inicial del entorno. Se repitió la prueba de integración con acceso local y pasó 1/1. Así, los 46 casos quedaron verificados entre ambas ejecuciones. Incluyen puntaje, restricciones, agrupación, reproducibilidad, física, métricas y persistencia.

**Comparación:** se ejecutó backend/scripts/compareStrategies.js con umbrales predeterminados, sin recalibrarlos. Usa 41 seeds de prueba (42026 y 1001-1040), turnos nominales de 3 horas, rutas estimadas, preferencias vacías y cierre de entregas activas. No usa OSRM ni demanda real.

| Promedio en 41 escenarios | Baseline | SmartCourier |
|---------------------------|----------|--------------|
| Ganancia neta             | \$394.03 | \$555.88     |
| Km recorridos             | 105.59   | 83.99        |
| Entregas completadas      | 6.12     | 6.76         |

SmartCourier obtuvo mayor ganancia neta en **40 de 41** escenarios y una mejora promedio de **41.1%** frente a Baseline. Es evidencia del simulador bajo esas condiciones; no demuestra ese incremento en repartos reales. El script separa 40 seeds de entrenamiento para su modo --grid; ese ajuste no se ejecutó para este informe.

## Mapa de fuentes del repositorio

- [S1]** backend/src/agents/smartCourierAgent.js  
evaluateSmartCourier, computeScore y checkHardConstraints
- [S2]** backend/src/simulation/SimulationCore.js  
decideSmartCourier, tryBatch, settle, restoreAgent y eventos
- [S3]** backend/src/simulation/economics.js  
offerEconomics; costos, tráfico y cierre
- [S4]** backend/src/agents/batchEvaluator.js  
evaluateBatch y DEFAULT\_BATCH\_LIMITS
- [S5]** backend/src/agents/repositionEvaluator.js  
evaluateReposition
- [S6]** backend/src/agents/baselineAgent.js  
evaluateBaseline y DEFAULTS
- [S7]** backend/src/simulation/routePlanner.js  
buildTimeline y bestSequence; también OrderGenerator.js
- [S8]** backend/src/services/routing.service.js  
getRoute, estimateRoute y caché
- [S9]** backend/src/simulation/zones.js  
zoneDemandLevel; también simulatedTime.js
- [S10]** backend/src/simulation/SimulationEngine.js  
motor; services/simulation.service.js y user.service.js
- [S11]** backend/scripts/compareStrategies.js  
benchmark; backend/test/ y utils/metricsCalculator.js
- [S12]** frontend/src/components/DecisionInspectorModal.js  
computeWhatIf
- [S13]** optimization-service/main.py  
API OR-Tools; backend/src/controllers/baseline.controller.js